

Machine learning Assignment 1

George Brady

My data dataset ID is: # id:14-28--14

(a)(i) The figure 1 scatter plot is a plot of the first feature on the x axis (X_1) vs the second feature on the y axis (X_2). The plot points are 'o' and blue if the target values are -1 and '+' and green if the target values are $+1$. As you can see from the graph most of the -1 points are in the upper part of graph and most of the $+1$ points are in the lower part of the graph. There appears to be a u-shape which separates them which is a curved decision boundary.

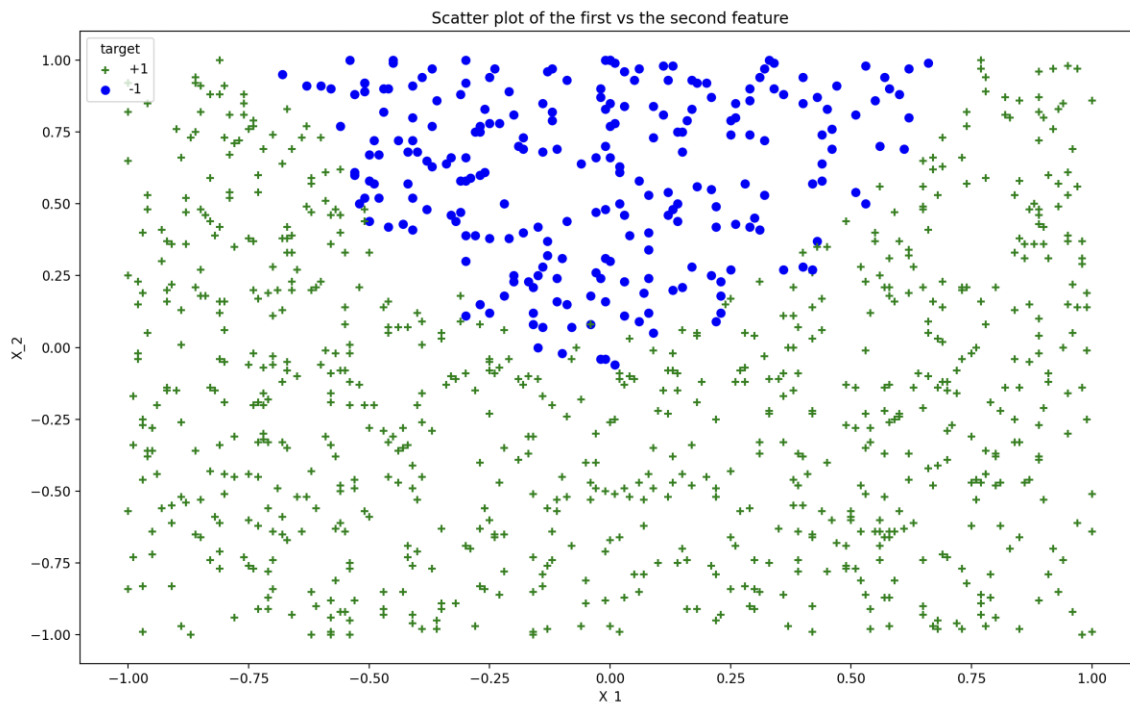


figure 1

(ii) I trained a logistic regression model on the data using the line model = `LogisticRegression(penalty=None, solver='lbfgs')`, I used `penalty=None` to make the model without L2 regularisation. The logistic regression's model's intercept is 2.170. The model's first parameter value is 0.047 and the model's second parameter value is -3.781 . This means that the formula for the prediction model is $\hat{y} = \text{sign}(2.170 + 0.047X_1 - 3.781X_2)$. The intercept being 2.170 is the model's base score, when the two features are set to 0 it being 2.170 means that the model is positive and when the features are smaller the output is more likely to be positive as this number is large, positive and does not change with features. The first feature having a parameter weight of 0.047, means that the first feature has little impact in the prediction formula. The

second feature having a parameter weight of -3.781 is a large negative weight. That means that having a larger second feature will significantly increase the likelihood that the prediction is a negative output. As the second parameter weight is much larger than the first parameter weight, the second feature has a much larger impact on the output of the prediction.

(iii) In figure 2 below you can see the values predicted to $+1$ as red 'x's and the values predicted to -1 as orange 'x's over the actual values that are -1 and $+1$, so that the predictions can be easily distinguished from the actual values. You can also see the decision boundary separating the predicted -1 value from the $+1$ values as a dotted line.

To get the equation for this line I started with the equation for the logistic regression's model $y = \text{intercept} + (\text{firstParameter})(X_1) + (\text{secondParameter})(X_2)$. The decision boundary is where the regression model is equal to 0 as if it is above 0 it is $+1$, and if it is below 0 it is -1 , so the formula is: $0 = \text{intercept} + (\text{firstParameter})(X_1) + (\text{secondParameter})(X_2)$. Solving this for X_2 we get: $(\text{secondParameter})(X_2) = -\text{intercept} - (\text{firstParameter})(X_1)$, which when we isolate X_2 gives us the equation: $X_2 = (-\text{intercept} - (\text{firstParameter})(X_1)) / \text{secondParameter}$.

This is why I used the code `dBoundary = (-intercept - (parameterWeight0*uniformX1Values)) / parameterWeight1` to get my decision boundary, and I plotted the line over a range of 100 evenly spaced X_1 values in the range of -1 to $+1$ using the line `plt.plot(uniformX1Values, dBoundary, 'k--', label = 'Decision boundary')` to get a black dotted line.

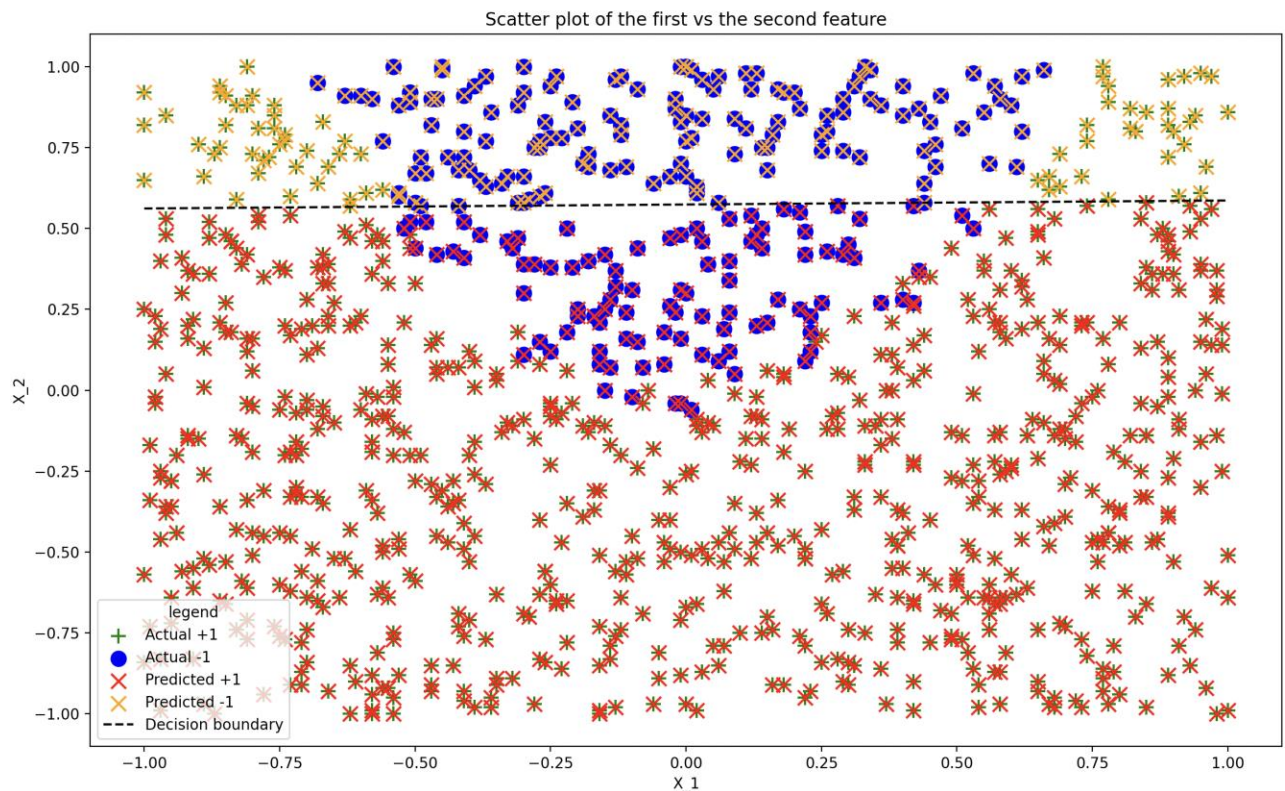


figure 2

(iv) As you can see in figure 2 a lot of the predictions are correct so in general the predictions are useful, but not all the predictions are correct there are a few predictions that are incorrect mainly close to the decision boundary. Not all of them are correct because the prediction model is based on a straight-line decision boundary and the actual outputs are separated by what seems to be a u-shaped curved decision boundary but there is still overlap between this curved boundary and even that wouldn't be a fully accurate prediction.

(b)(i) I trained three different linear SVM classifiers using the `linearSVC()` function in `sklearn` with three different C values and the results when I printed them out are:

- When $C=0.001$ the intercept=0.087 and the two parameters values are 0.0004 and -0.053
- When $C=1$ the intercept=0.729 and the two parameters values are 0.017 and -1.297
- When $C=100$ the intercept=0.73363413 and the two parameters values are 0.017 and -1.306

In these classifiers C controls the regularisation as it increases the model becomes less regularised and fits more to the data. You can see that when C is 0.001 the weights are very small as the model is heavily penalised for having big weights and is very

restricted. Once C increases to 1 the parameter value for the second feature gets much larger it goes to -1.297 making the second feature much more significant in the prediction, this increase is because the model penalises having larger weights less as C increases. Then When C increases to 100 you can see that the parameter values increase again but not as significantly as the last increase and this is because once C is large enough the model stops heavily regularising and it fits much more closely to the data. In all these cases the intercept increases as C goes up because the significance of the second feature increases.

Compared to the logistic regression model where the intercept= 2.170 and the two parameter values are 0.047 and -3.781 all of the classifiers follow the same pattern where the weight of the second feature is much more than the first but the logistic regression model has larger weights so will be effected more by increases in the second feature.

(ii) In figures 3, 4 and 5 below you can see the values predicted to +1 as red 'x's and the values predicted to -1 as orange 'x's over the actual values that are -1 and +1, so that the predictions can be easily distinguished from the actual values. You can also see the decision boundary separating the predicted -1 value from the +1 values as a black dotted line.

You can see that in figure 3 the line is above all of the values and is predicting that all of the values will be +1 this is because $C=0.001$ meaning that the model gets regularised heavily and is underfitted. When $C=1$ the boundary model moves down and separates the classes more accurately this is because the model is now less regularised and allows for larger weights. When $C=100$ it looks very similar to when $C=1$ and to the logistic regression model this is because the model is much less regularised and fits to the data to give the best prediction it can.

To get the equation for this line I started with the equation for the logistic regression's model $y = \text{intercept} + (\text{firstParameter})(X1) + (\text{secondParameter})(X2)$. The decision boundary is where the regression model is equal to 0 as if it is above 0 it is +1, and if it is below 0 it is -1, so the formula for it is: $0 = \text{intercept} + (\text{firstParameter})(X1) + (\text{secondParameter})(X2)$. Solving this for $X2$ gives us: $X2 = (-\text{intercept} - (\text{firstParameter})(X1)) / \text{secondParameter}$.

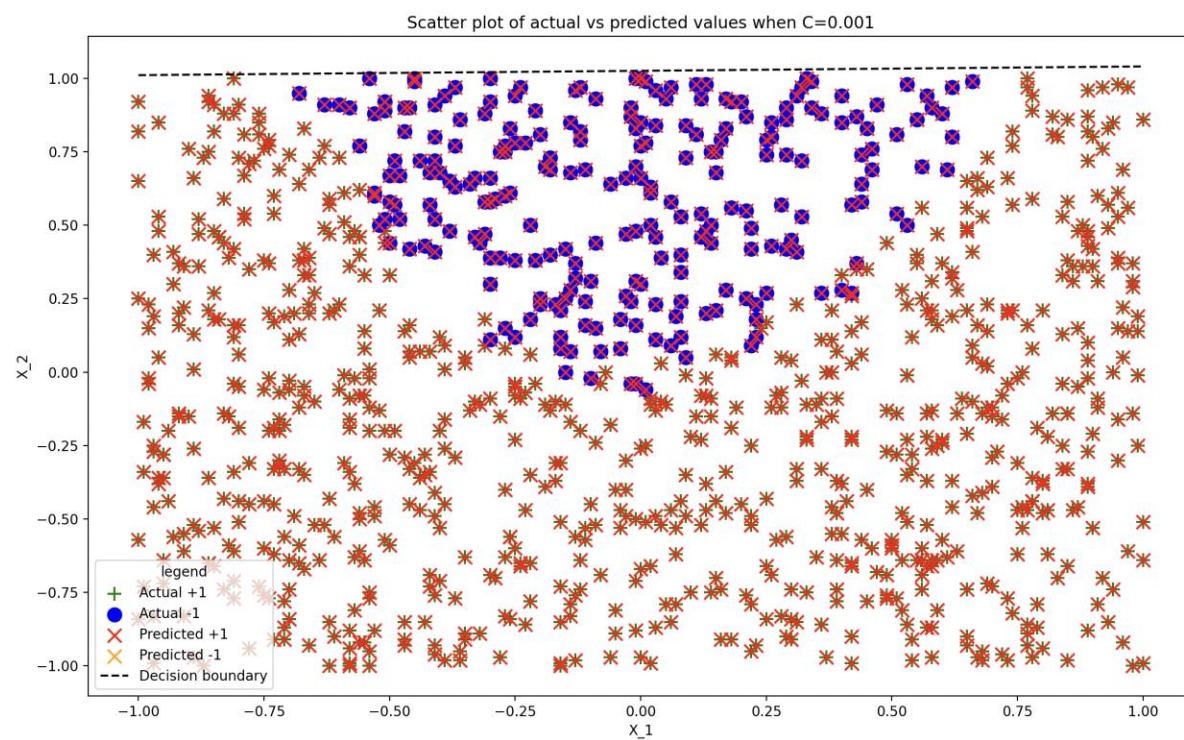


figure 3

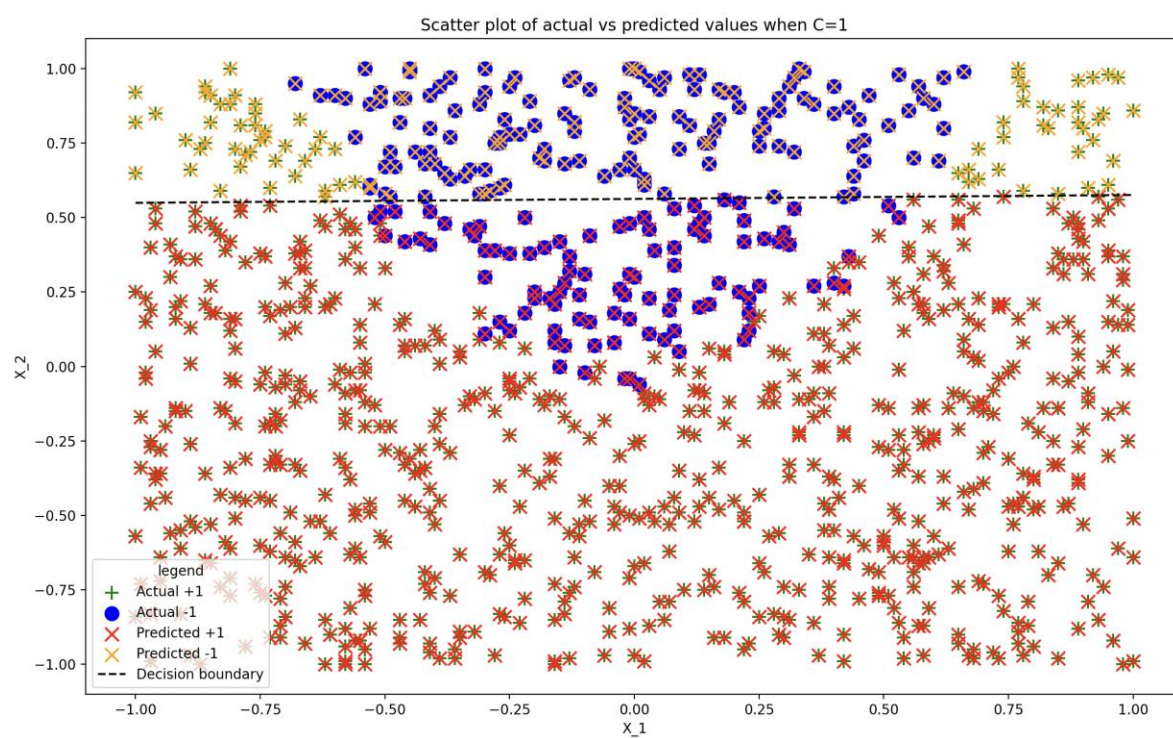


figure 4

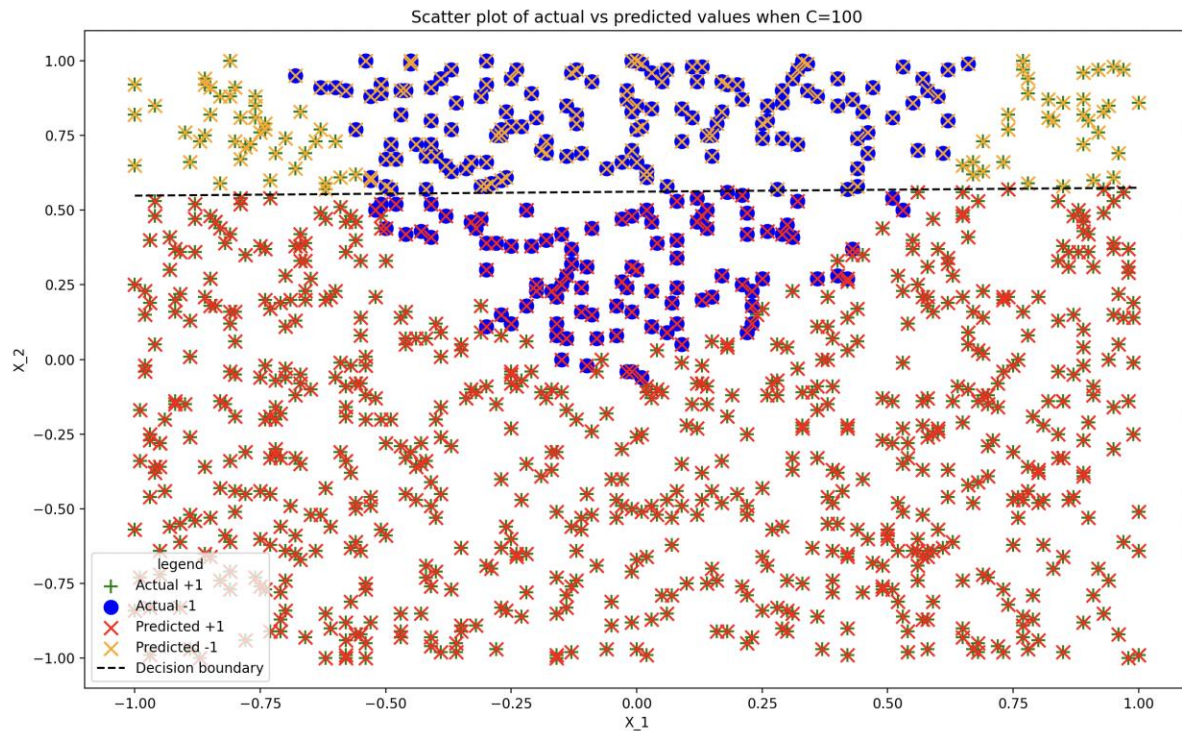


figure 5

(iii) The C value controls how much the model is regularised as C is increased the model becomes less regularised and maps to the actual output more accurately. This is because the L_2 penalty weight is equal to $1/C$. So, when C is small $1/C$ or L_2 is larger, this forces the weights of the features to be smaller and because of this the model's predictions are less accurate. And when the C value is large $1/C$ or the L_2 penalty weight is smaller making the model less regularised and allowing the weights of the features to be larger, this makes the model more accurate at predicting but risks overfitting the larger the C value gets.

(iv) In general, the SVM model parameters follow the same kind of pattern as the logistic regression in part (a) as they all have a highly weighted second feature compared to the first feature meaning the second feature has a much more significant effect on all of their outputs. The difference between the two models is that the weights in the logistic regression are much larger than in the SVM models and that is because C controls the regularisation of the model so that the weights can't be as large especially in the $C=0.001$ model where the weights are much smaller than the other two SVM models. The intercepts of the SVM models are also less than the intercept of the logistic regression model and that is because the SVM model uses support vectors to find the boundary and doesn't just fit the data exactly. In general, the predictions and decision boundary for the SVM models where $C=1$ and $C=100$ are very similar to the regression model in (a) but the predictions and decision boundary for the SVM models where

$C=0.001$ looks different with all of the predictions predicting that the output is +1 because the model is highly regularised.

(c)(i) For this part I trained the model on four inputs, the features X_1 , X_2 , X_1^2 and X_2^2 and I trained the model on these features using the following lines of code:

```
XWithSquared = np.column_stack((X1,X2,pow(X1,2),pow(X2,2)))
```

```
XSquaredmodel = LogisticRegression(penalty=None, solver='lbfgs').
```

This gave me the outputs intercept= -0.192, $X_1=0.036$, $X_2=-26.681$, $X_1^2=50.095$, $X_2^2=1.428$. The addition of squared values makes the decision boundary curved. X_2 is still large and will have a large impact linearly but now X_1^2 is large and will mean that X_1 will also have a large impact on the model now.

(ii) In figure 6 below you can see the values predicted to +1 as red 'x's and the values predicted to -1 as orange 'x's over the actual values that are -1 and +1, so that the predictions can be easily distinguished from the actual values. You can also see the decision boundary separating the predicted -1 value from the +1 values as a curved unbroken black line.

As a result of adding the squared features X_1^2 and X_2^2 the decision boundary in figure 6 is curved not straight anymore. You can see that the amount of predictions that match the actual data has increased with the curved boundary compared to the linear boundary. This is because this model is more accurate than a linear regression model for this data set, as the classes are separated by a curved relationship.

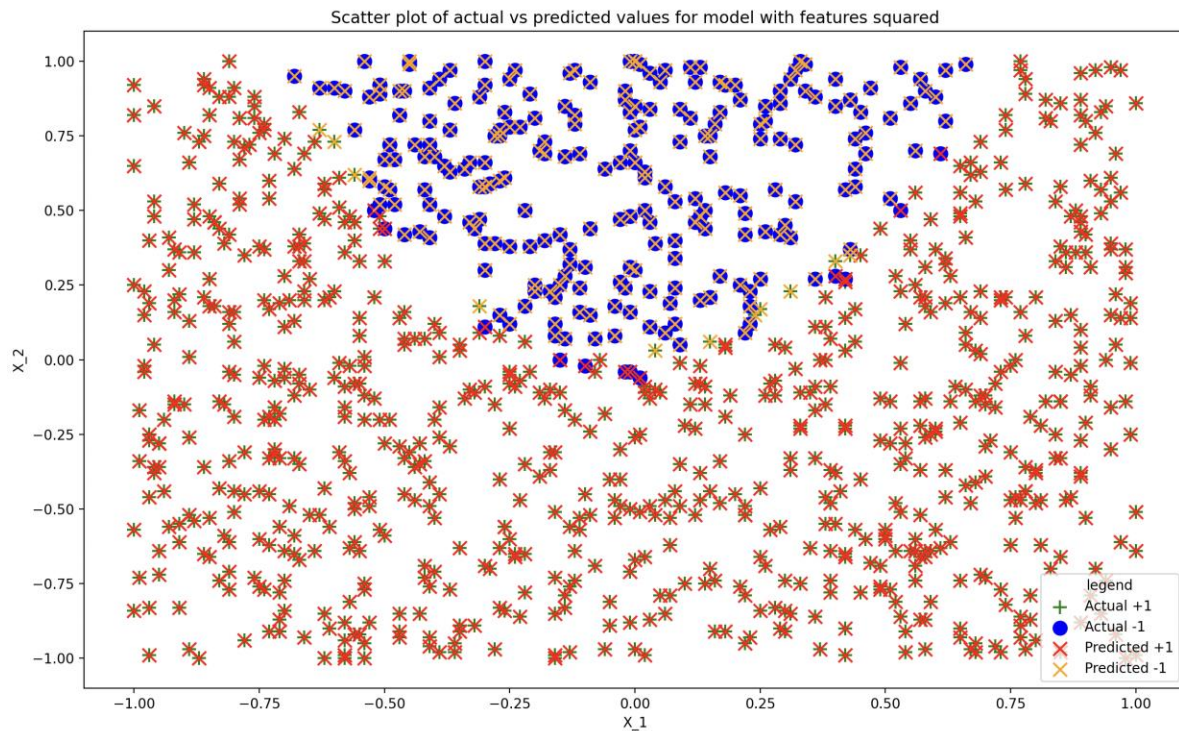


figure 6

(iii) The baseline predictor that I created which predicts +1, the most common result for this dataset every time is accurate 76.176% of the time. This is much less than our classifier which predicts the right output in the dataset 97.598% of the time. This means that the logistic regression classifier is much more accurate at predicting the outcome in this dataset.

(iv) In figure 7 below I have plotted the curved decision boundary for this classifier. I did this by making my formula $y = \text{intercept} + (\text{firstParameter})(X_1) + (\text{secondParameter})(X_2) + (\text{thirdParameter})(X_1^2) + (\text{fourthParameter})(X_2^2)$ into the quadratic formula $aX_2^2 + bX_2 + c$, so that it can be solved for X_2 . This meant $a = \text{fourthParameter}$, $b = \text{secondParameter}$ and $c = \text{firstParameter}(X_1) + \text{fourthParameter}(X_1^2) + \text{intercept}$. I could then solve for X_2 using the formulas $X_2 = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$, I tried both formulas and found that the correct one curve for my model was gotten by $X_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$.

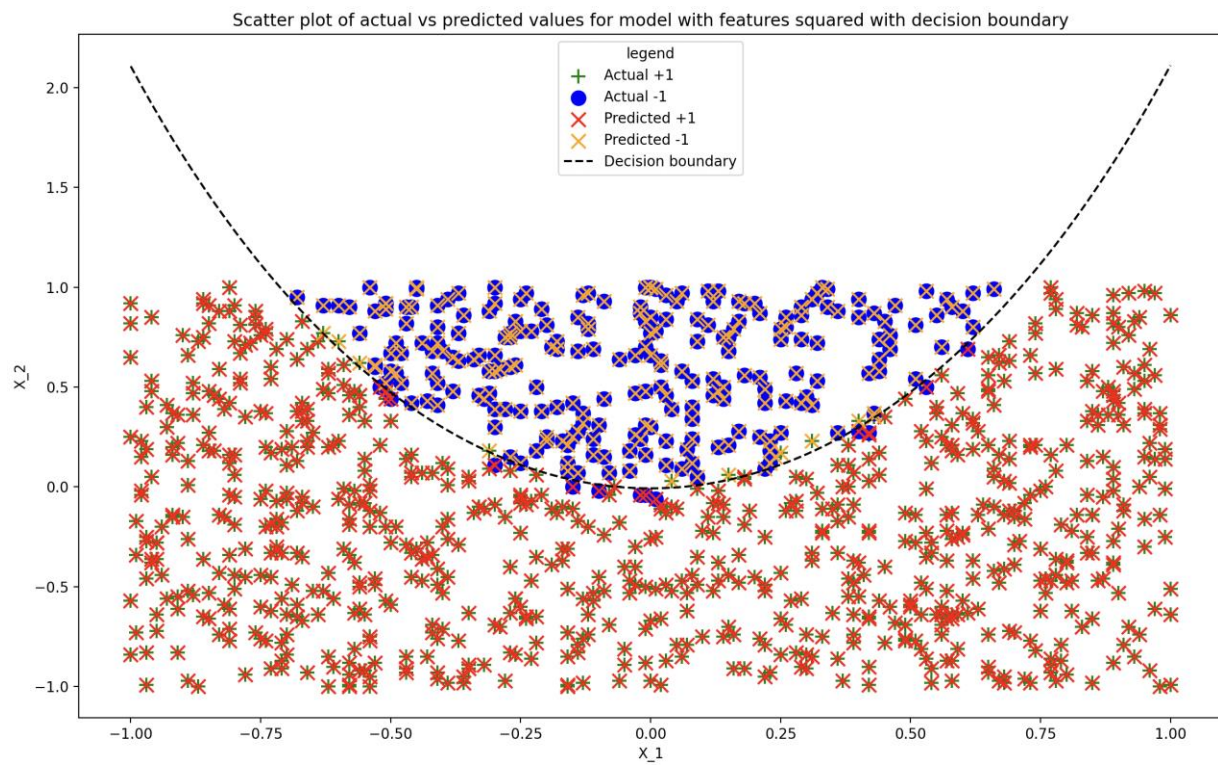


figure 7

Appendix of code:

```

assignment1.py > ...
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.svm import LinearSVC
6
7 df = pd.read_csv('week2.csv', header=None, skiprows=1) # tell it there are no titles so it
8 # can read in the first line and tell it
9 # to skip the ID line
10 print(df.head())
11 X=df.iloc[:,0]
12 X2=df.iloc[:,1]
13 X=np.column_stack((X1,X2))
14 y=df.iloc[:,2]
15
16 # (a)(i)
17 # loop through all of the y values and plot the x1 and x2 value
18 # depending on whether y is equal to 1 or -1
19 plt.scatter(X1[y==1], X2[y==1], color='green', marker='+', label='Actual +1', s=100)
20 plt.scatter(X1[y==1], X2[y==1], color='blue', marker='o', label='Actual -1', s=100)
21 # plot everything
22 plt.title('Scatter plot of the first vs the second feature')
23 plt.xlabel('X,1')
24 plt.ylabel('X,2')
25 plt.legend(title='target')
26 plt.show()
27
28 # (a)(ii)
29 model = LogisticRegression(penalty=None, solver='lbfgs')
30 model.fit(X, y)
31 # print the intercept and the coefficients
32 print('Intercept:', model.intercept_)
33 print('Coefficients:', model.coef_)
34
35 # (a)(iii)
36 # loop through all of the y values and plot the x1 and x2 value
37 # depending on whether y is equal to 1 or -1
38 plt.scatter(X1[y==1], X2[y==1], color='green', marker='+', label='Actual +1', s=100)
39 plt.scatter(X1[y==1], X2[y==1], color='blue', marker='o', label='Actual -1', s=100)
40 # get the y predictions based on the model and map the x values based on the
41 # y predictions using x markers so that they are visible
42 y_pred = model.predict(X)
43 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='red', marker='x', label='Predicted +1', s=100)
44 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='orange', marker='x', label='Predicted -1', s=100)
45
46 # assignment1.py > ...
47 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='orange', marker='x', label='Predicted -1', s=100)
48 # get the intercept and the parameter weights and map the boundary based on it
49 intercept = model.intercept_[0]
50 parameterWeights, parameterWeight = model.coef_[0]
51 uniformX1Values = np.linspace(-1, 1, 100)
52 dBoundary = (-intercept - (parameterWeight*uniformX1Values)) / parameterWeight1
53 # plot everything
54 plt.title('Scatter plot of the first vs the second feature')
55 plt.xlabel('X,1')
56 plt.ylabel('X,2')
57 plt.plot(uniformX1Values, dBoundary, 'k--', label='Decision boundary')
58 plt.legend(title='legend')
59 plt.show()
60
61 # (b)(i)
62 SVM0001model = LinearSVC(C=0.001)
63 SVM0001model.fit(X, y)
64 print('0.001 model intercept:', SVM0001model.intercept_)
65 print('0.001 model coefficient:', SVM0001model.coef_)
66
67 SVM1model = LinearSVC(C=1)
68 SVM1model.fit(X, y)
69 print('1 model intercept:', SVM1model.intercept_)
70 print('1 model coefficient:', SVM1model.coef_)
71
72 SVM10model = LinearSVC(C=100)
73 SVM10model.fit(X, y)
74 print('100 model intercept:', SVM10model.intercept_)
75 print('100 model coefficient:', SVM10model.coef_)
76
77 # (ii)
78 # for 0.001
79 plt.scatter(X1[y==1], X2[y==1], color='green', marker='+', label='Actual +1', s=100)
80 plt.scatter(X1[y==1], X2[y==1], color='blue', marker='o', label='Actual -1', s=100)
81 # get the y predictions based on the model and map the x values based on the
82 # y predictions using x markers so that they are visible
83 y_pred = SVM0001model.predict(X)
84 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='red', marker='x', label='Predicted +1', s=100)
85 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='orange', marker='x', label='Predicted -1', s=100)
86 # get the intercept and the parameter weights and map the boundary based on it
87 intercept = SVM0001model.intercept_[0]
88 parameterWeights, parameterWeight1 = SVM0001model.coef_[0]
89 uniformX1Values = np.linspace(-1, 1, 100)
90 dBoundary = (-intercept - (parameterWeight1*uniformX1Values)) / parameterWeight1
91 # plot everything
92 plt.title('Scatter plot of actual vs predicted values when C=0.001')
93 plt.xlabel('X,1')
94 plt.ylabel('X,2')
95 plt.plot(uniformX1Values, dBoundary, 'k--', label='Decision boundary')
96 plt.legend(title='legend')
97 plt.show()
98
99 # for 1
100 plt.scatter(X1[y==1], X2[y==1], color='green', marker='+', label='Actual +1', s=100)
101 plt.scatter(X1[y==1], X2[y==1], color='blue', marker='o', label='Actual -1', s=100)
102 # get the y predictions based on the model and map the x values based on the
103 # y predictions using x markers so that they are visible
104 y_pred = SVM1model.predict(X)
105 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='red', marker='x', label='Predicted +1', s=100)
106 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='orange', marker='x', label='Predicted -1', s=100)
107 # get the intercept and the parameter weights and map the boundary based on it
108 intercept = SVM1model.intercept_[0]
109 parameterWeights, parameterWeight1 = SVM1model.coef_[0]
110 uniformX1Values = np.linspace(-1, 1, 100)
111 dBoundary = (-intercept - (parameterWeight1*uniformX1Values)) / parameterWeight1
112 # plot everything
113 plt.title('Scatter plot of actual vs predicted values when C=1')
114 plt.xlabel('X,1')
115 plt.ylabel('X,2')
116 plt.plot(uniformX1Values, dBoundary, 'k--', label='Decision boundary')
117 plt.legend(title='legend')
118 plt.show()
119
120 # for 100
121 plt.scatter(X1[y==1], X2[y==1], color='green', marker='+', label='Actual +1', s=100)
122 plt.scatter(X1[y==1], X2[y==1], color='blue', marker='o', label='Actual -1', s=100)
123 # get the y predictions based on the model and map the x values based on the
124 # y predictions using x markers so that they are visible
125 y_pred = SVM10model.predict(X)
126 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='red', marker='x', label='Predicted +1', s=100)
127 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='orange', marker='x', label='Predicted -1', s=100)
128 # get the intercept and the parameter weights and map the boundary based on it
129 intercept = SVM10model.intercept_[0]
130 parameterWeights, parameterWeight1 = SVM10model.coef_[0]
131 uniformX1Values = np.linspace(-1, 1, 100)
132 dBoundary = (-intercept - (parameterWeight1*uniformX1Values)) / parameterWeight1
133 # plot everything
134 plt.title('Scatter plot of actual vs predicted values when C=100')
135 plt.xlabel('X,1')
136 plt.ylabel('X,2')
137 plt.plot(uniformX1Values, dBoundary, 'k--', label='Decision boundary')
138 plt.legend(title='legend')
139 plt.show()
140
141 # (c)(i)
142 # add the two X values squared to the regression
143 XWithSquared = np.column_stack((X1,X2,pow(X1,2),pow(X2,2)))
144 XSquaredmodel = LogisticRegression(penalty=None, solver='lbfgs')
145 XSquaredmodel.fit(XWithSquared, y)
146 print('Intercept for the 4 X value model:', XSquaredmodel.intercept_)
147 print('Coefficient for the 4 X value model:', XSquaredmodel.coef_)
148
149 # (ii)
150 plt.scatter(X1[y==1], X2[y==1], color='green', marker='+', label='Actual +1', s=100)
151 plt.scatter(X1[y==1], X2[y==1], color='blue', marker='o', label='Actual -1', s=100)
152 # get the y predictions based on the model and map the x values based on the
153 # y predictions using x markers so that they are visible
154 y_pred = XSquaredmodel.predict(XWithSquared)
155 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='red', marker='x', label='Predicted +1', s=100)
156 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='orange', marker='x', label='Predicted -1', s=100)
157 # get the intercept and the parameter weights and map the boundary based on it
158 intercept = XSquaredmodel.intercept_[0]
159 parameterWeights, parameterWeight1 = XSquaredmodel.coef_[0]
160 uniformX1Values = np.linspace(-1, 1, 100)
161 dBoundary = (-intercept - (parameterWeight1*uniformX1Values)) / parameterWeight1
162 # plot everything
163 plt.title('Scatter plot of actual vs predicted values for model with features squared')
164 plt.xlabel('X,1')
165 plt.ylabel('X,2')
166 plt.plot(uniformX1Values, dBoundary, 'k--', label='Decision boundary')
167 plt.legend(title='legend')
168 plt.show()
169
170 # (iii)
171 numberMinusOnes = sum(y==1)
172 numberPlusOnes = sum(y==1)
173 if numberPlusOnes > numberMinusOnes:
174     mostCommonClass = +1
175 else:
176     mostCommonClass = -1
177
178 baselinePrediction = sum(y==mostCommonClass) / len(y)
179 baselinePredictionPercent = baselinePrediction*100
180 print('The baseline prediction is, baselinePredictionPercent, %')
181 y_pred = XSquaredmodel.predict(XWithSquared)
182 XSquaredPrediction = sum(y==y_pred) / len(y)
183 XSquaredPredictionPercent = XSquaredPrediction*100
184 print('The X squares model prediction is, XSquaredPredictionPercent, %')
185
186 # (iv)
187 plt.scatter(X1[y==1], X2[y==1], color='green', marker='+', label='Actual +1', s=100)
188 plt.scatter(X1[y==1], X2[y==1], color='blue', marker='o', label='Actual -1', s=100)
189 # get the y predictions based on the model and map the x values based on the
190 # y predictions using x markers so that they are visible
191 y_pred = XSquaredmodel.predict(XWithSquared)
192 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='red', marker='x', label='Predicted +1', s=100)
193 plt.scatter(X1[y_pred==1], X2[y_pred==1], color='orange', marker='x', label='Predicted -1', s=100)
194 XValues = np.linspace(-1, 1, 100)
195 squaredIntercept = XSquaredmodel.intercept_[0]
196 firstParameter, secondParameter, thirdParameter, fourthParameter = XSquaredmodel.coef_[0]
197 # aX^2 + bX + c
198 # a = fourthParameter
199 # b = secondParameter
200 # c = firstParameter(X1) + fourthParameter(X^2) + intercept
201 # the formula for X2 is X2 = -b +/- square root of (b^2 - 4ac) / 2a
202 a = fourthParameter
203 b = secondParameter
204 c = (firstParameter * XValues) + (thirdParameter * (pow(XValues,2))) + squaredIntercept
205 X2Part1 = ((-b - np.sqrt(pow(b,2) - (4*a*c))) / (2*a)) this one is the wrong curve
206 X2Part2 = ((-b + np.sqrt(pow(b,2) - (4*a*c))) / (2*a))
207 plt.title('Scatter plot of actual vs predicted values for model with features squared with decision boundary')
208 plt.xlabel('X,1')
209 plt.ylabel('X,2')
210 #plt.plot(XValues, X2Part2, 'k--', label='Decision boundary') this one is the wrong curve
211 plt.plot(XValues, X2Part2, 'k--', label='Decision boundary')
212 plt.legend(title='legend')
213 plt.show()
214

```